Android - Unit 5

# Connect to the internet

User Interfaces 2

# Unit 5: Connect to the internet

(Vorige Canvas module)

1. Get data from the internet
    1. Connect to the internet
    2. Introduction to Coroutines in Kotlin
    3. Introduction to Coroutines in Android
    4. Introduction to HTTP/REST (dit ken je al)
    5. Get data from the internet (retrofit)
2. Load and display images from the internet
    1. Architecture: The data layer
    2a Add repository and Manual Dependency Injection
    2b Koin Dependency Injection (niet in Google cursus)
    3. Load and display images from the internet





# Get data from the internet (REST)



- Voeg netwerk permissie toe aan `AndroidManifest.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest ...>
    <uses-permission android:name="android.permission.INTERNET" />
    <application
        ...
        android:usesCleartextTraffic="true"
        ...>
        ...
    </application>
</manifest>
```

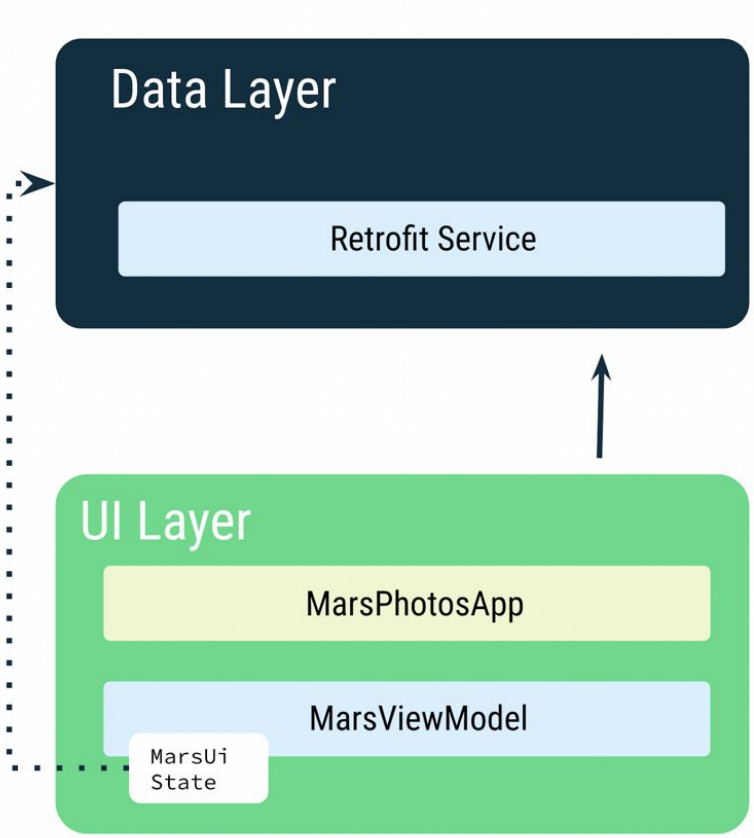
*Laat een app toelating vragen aan de telefoon gebruiker*

*Als je HTTP ipv HTTPS wil gebruiken (niet in codelab).*
*Test met* `http://android-kotlin-fun-mars-server.appspot.com`

# REST libraries



- REST met de [Retrofit bibliotheek]



# REST libraries



- REST met kotlin serialization en Retrofit bibliotheek

**Project** build.gradle.kts:

```
plugins {
 id("com.android.application") version "8.8.0" apply false
 id("com.android.library") version "8.8.0" apply false
 id("org.jetbrains.kotlin.android") version "2.1.0" apply false
 id("org.jetbrains.kotlin.plugin.compose") version "2.1.0" apply false
 id("org.jetbrains.kotlin.plugin.serialization") version "2.1.0" apply false
}


tasks.register("clean", Delete::class) {
 delete(rootProject.layout.buildDirectory)
}
```

Er zijn recentere library versies, we gebruiken die later in de Koin sectie

Licht andere syntax dan codelab, die geen deprecated warning geeft

# REST libraries



- REST met kotlin serialization en [Retrofit bibliotheek]

voeg toe aan **module** `build.gradle.kts`
plugins sectie:

```
id("org.jetbrains.kotlin.plugin.serialization")
```

dependencies sectie:

```
implementation("com.squareup.retrofit2:retrofit:2.11.0")
// Retrofit with Scalar Converter
// implementation("com.squareup.retrofit2:converter-scalars:2.11.0")
// retrofit with kotlin serialization converter
implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.8.0")
implementation("com.jakewharton.retrofit:retrofit2-kotlinx-serialization-converter:1.0.0")
implementation("com.squareup.okhttp3:okhttp:4.12.0")
```

# Retrofit stappenplan



1. Build een `Retrofit` connector
   - `Retrofit.Builder()`
   - `.addConverterFactory()`
     - scalarConverter: `JSON` over HTTP ⇔ `String in memory`
     - OF Kotlin serialization: `JSON` ⇔ `Kotlin objecten in memory`
   - `.baseUrl(BASE_URL)`
   - `.build()`

*Voor het connecteren naar een lokale json-server: volg de instructies in de projectopdracht!*

# Retrofit stappenplan



2. Maak een `Service` `interface` bv. `MarsApiService` met methoden om de webserver aan te spreken.
   - *suspendable functies met [Retrotfit REST annotatie]*
     - `@GET("users/list")`
     - `@POST("users/new")`
     - *...*
3. Maak een `singleton object` met een Retrofit implementatie van de service interface
   - later zullen we hiervoor dependency injection gebruiken
   - `by lazy`: delegeer voor initialisatie bij eerste gebruik

# Retrofit stappenplan



4. In de ViewModel
    - launch een coroutine via *viewModelScope*
    - Roep de `singleton` service aan en bewaar het resultaat in de ViewModel (mutable-)State)
5. Gebruik het ViewModel in de UI Composables
    - De state in het viewModel wordt asynchroon aangepast
        - ➢ recomposition triggert na einde coroutine

# Sealed Class/interface



- Kotlin sealed classes/interfaces beperken het aantal mogelijke subklassen
  - hier zijn de subklassen geïmplementeerd als inner classes

```
sealed interface MarsUiState {
 data object Loading : MarsUiState
 data class Success(val photos: String) : MarsUiState
 data object Error : MarsUiState
}
```

Loading is een Singleton subklasse van MarsUiState

Success Subklasse constructor



Te behalen Pathway 1:
https://developer.android.com/courses/pathways/android-basics-compose-unit-5-pathway-1

# Unit 5: Connect to the internet

(Vorige Canvas module)

1. Get data from the internet
    1. Connect to the internet
    2. Introduction to Coroutines in Kotlin
    3. Introduction to Coroutines in Android
    4. Introduction to HTTP/REST (dit ken je al)
    5. Get data from the internet (retrofit)
2. Load and display images from the internet
    1. Architecture: The data layer
    2a Add repository and Manual Dependency Injection
    2b Koin Dependency Injection (niet in Google cursus)
    3. Load and display images from the internet



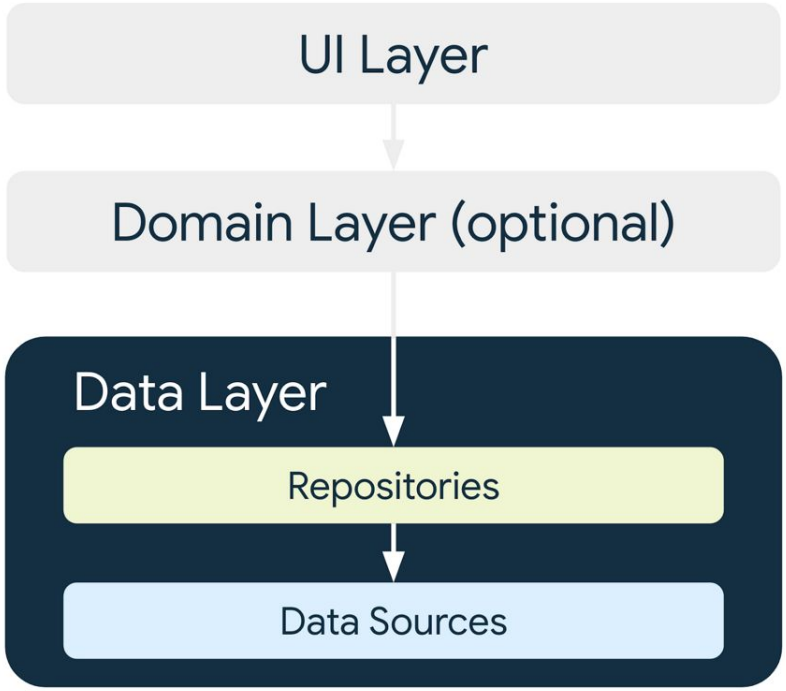


# Add repository



Repository setup (data layer)

- [separation of concerns]
- Naamgeving : “<type of data>”`Repository`
- Alle toegang naar backend systemen gaat via Repository (interface)



# Manual Dependency Injection

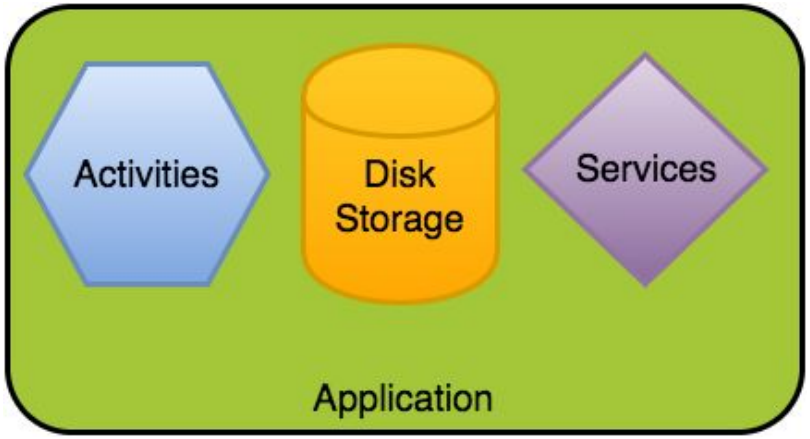


- Je code is enkel afhankelijk van interfaces
- Geef de nodige implementaties door als parameters → `inversion of control`
- Gebruik een container waarin dependencies geïnitialiseerd worden.
- Instantieer deze container als éérste d.m.v. een custom `Application class`
  - Niet nodig als je geen specifieke initialisaties nodig hebt. Je gebruikt dan automatisch de default Application

# Manual Dependency Injection



- Application Class
  - basisklasse binnen elke Android app die alle andere componenten zoals activities en services bevat.
  - `Application` wordt geïnitialiseerd vóór elke andere klasse
  - Kan geconfigureerd worden in `AndroidManifest.xml`



# Manual Dependency Injection



- Application Class
  - Op te halen via `APPLICATION_KEY`

```
val application = (this[APPLICATION_KEY] as MarsPhotosApplication)
```

  - Geeft toegang tot je container
  - Dependencies doorgeven aan bv. `viewModel` met `viewModelFactory`

# Manual Dependency Injection



- `ViewModelProvider`:  manier om ViewModels te maken en te verkrijgen
- `ViewModelFactory`: maakt instances van ViewModels  met custom constructors.
    - Bevat `initializer` extension om extra parameters mee te geven als constructor (`CreationExtras`)

```
Surface(
            // ...
        ) {
            val marsViewModel: MarsViewModel =
                        viewModel(factory = MarsViewModel.Factory)
            // ...
        }
```

# PW2 CodeLab 2 Add repository and Manual Dependency Injection



- ~~7. Get setup for local tests~~
- ~~8. Create fake data and dependencies for tests~~
- ~~9. Write a repository test~~
- ~~10. Write a ViewModel test~~




*Extra voorbeeld met manual dependency injection en json-server: Todo app*

# Unit 5: Connect to the internet

(Vorige Canvas module)

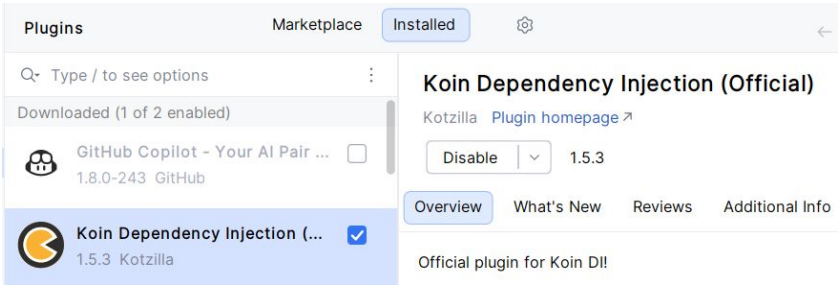
1. Get data from the internet
    1. Connect to the internet
    2. Introduction to Coroutines in Kotlin
    3. Introduction to Coroutines in Android
    4. Introduction to HTTP/REST (dit ken je al)
    5. Get data from the internet (retrofit)
2. Load and display images from the internet
    1. Architecture: The data layer
    2a Add repository and Manual Dependency Injection
    2b Koin Dependency Injection (niet in Google cursus)
    3. Load and display images from the internet





# Koin Dependency Injection

- Koin is een Kotlin Dependency Injection framework: https://insert-koin.io
- koin-android ondersteunt Android componenten en scopes
- Compile time dependency checking
  - ⇔ Spring framework doet DI enkel at runtime
- Er is een Koin Dependency Injection plugin voor Android studio
  - je moet inloggen om te activeren :(



# Koin: Gradle configuratie

configuratie met libs.versions.toml in slide notes

project build.gradle.kts

```
plugins {// … andere plugins
    id("com.google.devtools.ksp") version "2.3.7" apply false
}
```

Kotlin compiler plugin

module build.gradle.kts

```
plugins {// … andere plugins
    id("com.google.devtools.ksp")
}// ...meer config
dependencies {// … andere dependencies
    implementation("io.insert-koin:koin-core:4.2.1")
    implementation("io.insert-koin:koin-android:4.2.1")
    implementation("io.insert-koin:koin-androidx-compose:4.2.1")
    implementation("io.insert-koin:koin-annotations:2.3.1")
    ksp("io.insert-koin:koin-ksp-compiler:2.3.1")
}
ksp{
 arg("KOIN_CONFIG_CHECK", "true")
}
```

# Koin: Definieer injecties

- `@Module`
  - Groepeert DI definities

## Definition Types

*Wij gebruiken deze*

| TYPE | DSL | ANNOTATION | LIFECYCLE | USE CASE |
|---|---|---|---|---|
| Singleton | `single()` | `@Singleton` `@Single` | One instance for app lifetime | Services, repositories, databases |
| Factory | `factory()` | `@Factory` | New instance each time | Presenters, use cases, stateful objects |
| Scoped | `scoped()` | `@Scoped` | One instance per scope | Activity-bound, session-bound objects |
| ViewModel | `viewModel()` | `@KoinViewModel` | Android ViewModel lifecycle | ViewModels |

# Koin: Definieer injecties

- Maak een package **di** met **AppModule.kt** aan
  - definieer te injecteren implementaties
  - vergelijkbaar met Spring `@Configuration`

di/AppModule.kt

```kotlin
package com.example.marsphotos.di
import org.koin.core.annotation.*
// more imports
@Module
@ComponentScan("com.example.marsphotos")
class AppModule {

  private val BASE_URL = "https://android-kotlin-fun-mars-server.appspot.com/"

  @Single
  fun provideRetrofit(): Retrofit {
       // … return je geconfigureerde retrofit.Builder
  }

  @Single
  fun provideMarsApiService(retrofit: Retrofit): MarsApiService =
    retrofit.create(MarsApiService::class.java) // kotlin compacte functie notatie

}
```

Scan voor koin annotaties onder com.example.marsphotos (default: package van deze klasse)

Injectie van Retrofit parameter



# Koin: Definieer injecties

- Je kan ook te injecteren implementaties op de klasse definiëren (ComponentScan)
  - vergelijkbaar met Spring `@Component`

data/MarsPhotoRepository

```
package com.example.marsphotos.data
// imports weggelaten

interface MarsPhotoRepository {
 suspend fun getMarsPhotos(): List<MarsPhoto>
}

@Single
class RestMarsPhotoRepository(val apiService: MarsApiService) :
MarsPhotoRepository {
  override suspend fun getMarsPhotos(): List<MarsPhoto> {
    return apiService.getPhotos()
  }
}
```

We hebben de constructor aangepast om de in de module gedefinieerde MarsApiService te ontvangen



# Koin: injectie en ViewModel

- Annoteer je ViewModels, dependencies worden geïnjecteerd

```
@KoinViewModel
class MyViewModel(private val repository:MyRepository): ViewModel()
{    // ViewModel code here }
```

- In je `@Composable`, vervang ~~`ViewModel()`~~ door `koinViewModel()` voor het ophalen van je viewModel

```
import org.koin.compose.viewmodel.koinViewModel
// other imports
@Composable
fun MyComposable(viewModel: MyViewModel = koinViewModel()) {
    // Your Composable content here
}
```

  - Je kan je viewModel ook in je Activity aanmaken met `by viewModel()`



Voorbeeld van de Todo retrofit demo met Koin (koin branch)

# Koin starten

- Maak een Application()  klasse. Voorbeeld:

```
package com.example.marsphotos

import org.koin.ksp.generated.*  //other imports omitted

class MarsPhotoApplication:Application() {
 override fun onCreate() {
  super.onCreate()
  startKoin{
   androidLogger()
   androidContext(this@MarsPhotoApplication)
   modules(AppModule().module)
  }
 }
}
```

# Koin starten

- Pas `AndroidManifest.xml` aan om van de Application class te starten (ipv .MainActivity)

```
<manifest ...>
  <!--...-->
  <application android:name=".MarsPhotosApplication"...>
    <!--...-->
  </application>
</manifest>
```

. is relatief tov android.namespace in module build.gradle.kts

# Unit 5: Connect to the internet

(Vorige Canvas module)

1. Get data from the internet
    1. Connect to the internet
    2. Introduction to Coroutines in Kotlin
    3. Introduction to Coroutines in Android
    4. Introduction to HTTP/REST (dit ken je al)
    5. Get data from the internet (retrofit)
2. Load and display images from the internet
    1. Architecture: The data layer
    2a Add repository and Manual Dependency Injection
    2b Koin Dependency Injection (niet in Google cursus)
    3. Load and display images from the internet





# Internet images ophalen en tonen



- opslaan, decoderen, tonen

  → “expensive”

  → Background thread

- We gebruiken Coil (v2 !! ) compose library
  - URL van image

COIL

```
AsyncImage(
    model = "https://example.com/image.jpg",
    contentDescription = “alt-text”,
)
```

*Simpel, maar weinig controle*

# COIL ImageRequest



We geven een ImageRequest mee
→ meer controle over gedrag

```
AsyncImage(
    model = ImageRequest.Builder(
                context = LocalContext.current)
            .data(photo.imgSrc)
            .crossfade(true)
            .build(),

    contentDescription =  stringResource(R.string.mars_photo)
)
```

Crossfade als de image geladen is

# COIL AsyncImage



- Extra Parameters:
    - Error → Request mislukt
    - Placeholder → 'loading'

```
AsyncImage(
    ...
    error = painterResource(R.drawable.ic_broken_image),
    placeholder = painterResource(R.drawable.loading_img),
    ...
)
```

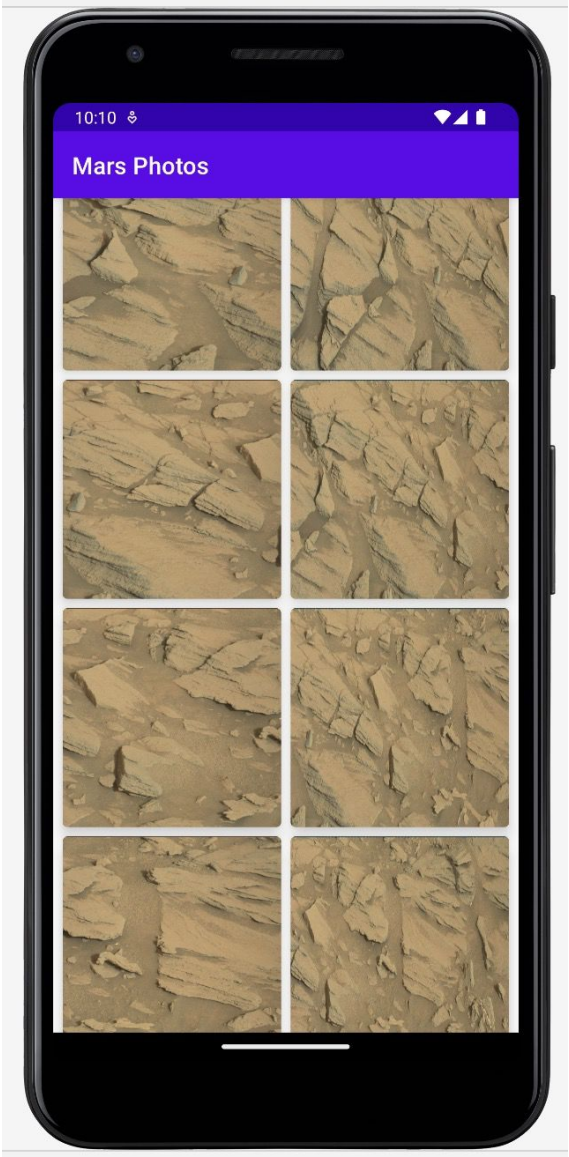
Voorbeeld van de MarsPhoto Oefening met Koin

(en Coil zie volgende pathway): koin-coil branch

# PW2 Lab 3 Display Images



- `LazyVerticalGrid`
- ~~(5. Update the ViewModel test)~~



# PW2 Oefeningen



- Practice: Build amphibians app
- Project: Create a Bookshelf app